

Código de Golay extendido (24, 12) — Parte B

Trabajo Práctico 2 · Curso de FEC 2026 · Fundación Fulgor

Villar, Pedro

El modelo de referencia está en el paquete `golay/` y usa la librería `fec_algebra` del TP1. Las tablas de este informe las genera `gen_parte_b.py` corriendo el modelo.

1 Ejercicio 4 — $GF(2^m)$ con $m = 1$

Se instancia el campo del TP1 con $m = 1$ y $P(x) = x + 1$: `GF(m=1, primitive_poly=0b1)`. Queda un campo de dos elementos con estas tablas:

$+$	0	1	$\cdot$	0	1	a	a^{-1}
0	0	1	0	0	0	0	—
1	1	0	1	0	1	1	1

La suma es XOR y el producto es AND, o sea que es $GF(2)$. No hubo que modificar la librería. El producto con $m = 1$ hace una sola iteración y da $a \wedge b$, y el inverso se calcula como $a^{2^m-2} = a^0 = 1$, que es correcto porque el único elemento invertible es el 1. El 0 tira `GFZeroDivisionError`.

Lo que faltaba era el álgebra de matrices, que está en `golay/linalg.py`: `vec_mat` (producto vector-matriz), `mat_mat` (producto matriz-matriz) y `weight` (peso de Hamming), más algunas auxiliares (`transpose`, `identity`, `hstack`, `vec_add`). Sirven para cualquier $GF(2^m)$, no solo para $GF(2)$.

2 Ejercicio 5 — Construcción y verificación

B se carga con las doce filas del enunciado y el resto se arma con `linalg`: $G = \text{hstack}(I, B)$, $H = \text{hstack}(B, I)$ y $H^T = \text{transpose}(H)$. Con los productos matriciales se verifica que $B^T = B$, $B^2 = I$ y $GH^T = 0$, igual que en la Parte A.

Codificando los 4096 mensajes con $v = m \cdot G$ y contando pesos:

peso w	0	8	12	16	24	total
modelo	1	759	2576	759	1	4096
enunciado	1	759	2576	759	1	4096

Coincide con la tabla del Ejercicio 2, y el peso mínimo no nulo da $d_{\min} = 8$.

3 Ejercicio 6 — Codificador y decodificador

El codificador es $v = m \cdot G$ con `vec_mat`. Para $m = \text{A5C}$ da $v = \text{A5C9A5}$, igual que a mano.

El decodificador tiene una función por cada submódulo del RTL de la Parte C, así después cada `testbench` compara contra su función:

RTL	modelo	etapa
<code>golay_syndrome</code>	<code>syndrome</code>	1
<code>golay_mult_b</code>	<code>mult_b</code>	2
<code>popcount12</code>	<code>popcount12</code>	2
<code>golay_row_search</code>	<code>row_search</code>	2
<code>golay_err_gen</code>	<code>err_gen</code>	3
<code>golay_correct</code>	<code>correct</code>	3

`decode()` llama a esas funciones en el orden del pipeline. Como `golay_correct` solo recibe `i_rx` e `i_err`, `o_corrected` es simplemente $e \neq 0$. Cuando `row_search` no encuentra fila, `o_idx` y `o_res` no importan.

3.1 Tabla de síndromes

La tabla se arma recorriendo todos los errores de peso ≤ 4 , calculando su síndrome con H y guardando, para cada síndrome, los patrones de peso mínimo:

peso del líder	líderes	cosets	
0	1	1	corregible
1	1	24	corregible
2	1	276	corregible
3	1	2024	corregible
4	6	1771	no corregible
4096			

Los 2325 síndromes con un solo líder se corrigen. Los 1771 restantes tienen seis líderes de peso 4 y no hay forma de elegir uno, así que se declaran no corregibles. El algoritmo de cuatro casos no necesita esta tabla, así que se usa como segundo decodificador para comparar resultados, y los dos dan lo mismo.

4 Ejercicio 7 — Caracterización

4.1 Errores de peso ≤ 4

Se decodifican todos los patrones de error de peso 0 a 4 sobre la palabra nula (alcanza con esa porque el síndrome solo depende de e):

peso de e	0	1	2	3	4
patrones	1	24	276	2024	10626
corregidos bien	1	24	276	2024	0
decodificados a otra palabra	0	0	0	0	0
detectados como no corregibles	0	0	0	0	10626

Da igual que la tabla del enunciado. Nunca se decodifica a una palabra equivocada: o corrige bien o marca que no puede.

4.2 Estímulo para la Parte C

No se generan archivos de vectores: los testbenches de cocotb importan `golay/stimulus.py` y generan el estímulo ahí mismo, como en la Unidad 3. Cada generador devuelve diccionarios con los nombres de los puertos del módulo:

```
for vec in mult_b_vectors():
    dut.i_vec.value = vec["i_vec"]
    await Timer(2, "ns")
    assert int(dut.o_vec.value) == vec["o_vec"]
```

Los módulos de 12 bits se prueban con las 4096 entradas. El decodificador completo tiene 2^{24} entradas posibles, así que se usa una muestra: las 4096 palabras código, todos los errores de peso 1 a 3 sobre cinco palabras, y parte de los de peso 4.

módulo	casos		caso	vectores	condición
golay_mult_b	4096	exhaustivo	1	5586	$w(s) \leq 3$
popcount12	4096	exhaustivo	2	4740	$\exists i : w(s \oplus b_i) \leq 2$
golay_row_search	4096	exhaustivo	3	1430	$w(q) \leq 3$
golay_encoder	4096	exhaustivo	4	3960	$\exists i : w(q \oplus b_i) \leq 2$
golay_syndrome	6420	palabras código + errores	5	110	no corregible
golay_decoder	15826	muestra, cubre los 5 casos			

5 Preguntas

5.1 ¿Por qué el síndrome depende solo del error?

Con $r = v + e$:

$$s = r H^T = v H^T + e H^T = e H^T,$$

porque $v = m G$ y $G H^T = 0$. El mensaje desaparece de la cuenta.

5.2 ¿Por qué $d_{\min} = 8$ detecta 4 errores pero no los corrige?

Detecta: si e pesa 4, $v + e$ no puede ser otra palabra código, porque la diferencia entre dos palabras código pesa al menos 8. Entonces el síndrome no es cero y el error se nota.

No corrige: para corregir t errores hace falta $d_{\min} \geq 2t + 1$, y con $d_{\min} = 8$ eso da $t = 3$. En la tabla de síndromes se ve por qué: un error de peso 4 cae en un síndrome que comparten seis patrones de peso 4, todos igual de probables, y no hay forma de saber cuál fue. Por eso el decodificador llega al caso 5 y levanta `o_uncorrectable`, que es lo que pasó con r_3 en la Parte A.